

Project User Guide & Technical Manual

Reference Guide: Operations, Setup, and Customization Details

1. Technical Architecture & File Map

This repository is organized as a modular, production-ready data science pipeline:

Directory/File	Functional Purpose
data/raw_sales_data.csv	Messy raw transactional input dataset with injected anomalies.
data/cleaned_sales_data.csv	Sanitized, standardised output dataset ready for analysis and BI tools.
scripts/generate_data.py	Data generator script used to programmatically compile sales and anomalies.
scripts/sales_analysis.py	Core pipeline script executing cleaning, EDA, visual plot generation, and ML forecasting.
visualizations/	Directory containing the 5 high-resolution exported PNG analysis charts.
sales_analysis.ipynb	Interactive Jupyter Notebook providing a step-by-step graphical pipeline interface.
requirements.txt	Pip dependency list containing required Python packages.

2. Setting up the Local Environment

We recommend executing this project in an isolated virtual environment to prevent package conflicts:

```
# Step 1: Open project directory
cd c:\Users\siddh\Desktop\andro\sales-insights-project

# Step 2: Create a virtual environment
python -m venv venv

# Step 3: Activate virtual environment
# On Windows PowerShell:
.\venv\Scripts\Activate.ps1
# On Mac/Linux:
source venv/bin/activate

# Step 4: Install dependencies
pip install -r requirements.txt
```

3. Running the Analysis Pipeline

Option A: Run the Script (Production)

Execute the primary analysis script from your terminal to clean the raw dataset, output full mathematical statistics, and export all 5 charts at once:

```
python scripts/sales_analysis.py
```

Option B: Run the Jupyter Notebook (Interactive)

Launch the Jupyter interface to run and inspect individual analysis cells chronologically:

```
jupyter notebook sales_analysis.ipynb
```

4. How to Use Your Own Sales Data

To run this pipeline on a real business's sales transactions, follow these steps:

1. Ensure your custom CSV file contains the following columns with matching casing: **Date, Quantity, Unit_Price, Category, Region, Customer_Segment, Product_Name, Transaction_ID, Customer_ID**.
2. Save your file, name it **raw_sales_data.csv**, and overwrite the existing file in the **data/** directory.
3. Open your terminal and run **python scripts/sales_analysis.py**.

The script will automatically clean your custom dataset, execute aggregations, and overwrite the visualizations with your real business data!

5. Presentation & Delivery Pitch Tips

When delivering this project to a freelance client or discussing it in interviews, highlight these three technical achievements:

- **Temporal Data Integrity:** Emphasize that your Machine Learning train-test split was executed chronologically, not randomly. Random splitting on time-series data leaks future information into training sets, rendering model results invalid in real-world applications.
- **Product-Specific Imputation:** Point out that missing or outlier unit prices were imputed dynamically using the median price of that specific product, rather than a generic dataset-wide average.
- **Business KPI Impact:** Frame your technical steps in terms of business metrics—such as optimizing inventory control and inventory carrying costs using the 4-week sales forecast.